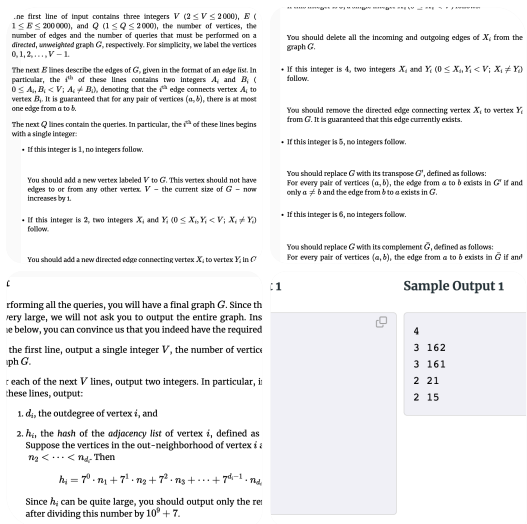

.ne first line of input contains three integers V ($2 \le V \le 2\,000$), E ($1 \le E \le 200\,000$), and Q ($1 \le Q \le 2\,000$), the number of vertices, the number of edges and the number of queries that must be performed on a *directed, unweighted* graph G, respectively. For simplicity, we label the vertices $0, 1, 2, \ldots, V-1$.

The next E lines describe the edges of G, given in the format of an *edge list*. In particular, the ith of these lines contains two integers A_i and B_i ($0 \le A_i, B_i < V$; $A_i \ne B_i$), denoting that the ith edge connects vertex A_i to vertex B_i. It is guaranteed that for any pair of vertices (a, b), there is at most one edge from a to b.

The next Q lines contain the queries. In particular, the ith of these lines begins with a single integer:

- If this integer is 1, no integers follow.

  You should add a new vertex labeled V to G. This vertex should not have edges to or from any other vertex. V – the current size of G – now increases by 1.

- If this integer is 2, two integers X_i and Y_i ($0 \le X_i, Y_i < V$; $X_i \ne Y_i$) follow.

  You should add a new directed edge connecting vertex X_i to vertex Y_i in G

You should delete all the incoming and outgoing edges of X_i from the graph G.

- If this integer is 4, two integers X_i and Y_i ($0 \le X_i, Y_i < V$; $X_i \ne Y_i$) follow.

  You should remove the directed edge connecting vertex X_i to vertex Y_i from G. It is guaranteed that this edge currently exists.

- If this integer is 5, no integers follow.

  You should replace G with its transpose G^T, defined as follows:
  For every pair of vertices (a, b), the edge from a to b exists in G^T if and only a $\ne$ b and the edge from b to a exists in G.

- If this integer is 6, no integers follow.

  You should replace G with its complement $\bar{G}$, defined as follows:
  For every pair of vertices (a, b), the edge from a to b exists in $\bar{G}$ if and

rforming all the queries, you will have a final graph G. Since th ery large, we will not ask you to output the entire graph. Ins e below, you can convince us that you indeed have the required

the first line, output a single integer V, the number of vertic ph G.

each of the next V lines, output two integers. In particular, i these lines, output:

1. d_i, the outdegree of vertex i, and
2. h_i, the *hash* of the *adjacency list* of vertex i, defined as Suppose the vertices in the out-neighborhood of vertex i $n_2 < \cdots < n_{d_i}$. Then

$$h_i = 7^0 \cdot n_1 + 7^1 \cdot n_2 + 7^2 \cdot n_3 + \cdots + 7^{d_i - 1} \cdot n_{d_i}$$

Since h_i can be quite large, you should output only the re after dividing this number by $10^9 + 7$.

1

Sample Output 1

```
4
3 162
3 161
2 21
2 15
```

Solve in Python efficiently and concisely

Thought for 3m 30s ›

Use a bitset adjacency matrix with two lazy flags:

- `rev` : current graph is transposed
- `inv` : current graph is complemented

Let `B` be the stored raw graph. For `i != j` :

$$G(i,j) = B(a,b) \oplus inv$$

where `(a,b) = (i,j)` normally, and `(a,b) = (j,i)` if `rev=1` .

This makes:

- transpose: just `rev ^= 1`
- complement: just `inv ^= 1`

For edge add/remove, convert the requested visible edge value back to the raw value and update one bit.

For:

- **add isolated vertex**
- **delete all incident edges of x**

all affected visible edges become **absent**, so the corresponding raw bits must all become `inv` . This is independent of `rev` , so both operations are just `O(V)` .

To make transpose cheap at the end, keep both row bitsets and column bitsets.

Complexity

- Build: `O(E)`
- Queries: `O(1)` for types `2/4/5/6`, `O(V)` for types `1/3`
- Final output: `O(V + number_of_final_edges)` for hashing
- With `V_final <= 4000`, this is easily fast enough.

```
import sys

MOD = 10**9 + 7

def solve() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    it = iter(data)

    n = next(it)
    e = next(it)
    q = next(it)

    rows = [0] * n   # raw outgoing bitsets
    cols = [0] * n   # raw incoming bitsets

    for _ in range(e):
        a = next(it)
        b = next(it)
        rows[a] |= 1 << b
        cols[b] |= 1 << a

    rev = 0   # lazy transpose
    inv = 0   # lazy complement

    for _ in range(q):
        t = next(it)

        if t == 1:
            # add isolated vertex n
            if inv:
                old = n
                bit = 1 << old
                mask = bit - 1          # bits [0..old-1] = 1
                rows.append(mask)       # new vertex -> old vertices are a
                cols.append(mask)       # old vertices -> new vertex likew
                for i in range(old):
                    rows[i] |= bit
                    cols[i] |= bit
            else:
                rows.append(0)
                cols.append(0)
            n += 1
```

```
        elif t == 2:
            # add edge x -> y in visible graph
            x = next(it)
            y = next(it)
            r, c = (x, y) if not rev else (y, x)
            want = 1 ^ inv
            if want:
                rows[r] |= 1 << c
                cols[c] |= 1 << r
            else:
                rows[r] &= ~(1 << c)
                cols[c] &= ~(1 << r)

        elif t == 3:
            # delete all incoming and outgoing edges of x in visible graph
            x = next(it)
            bit = 1 << x

            if inv:
                full = (1 << n) - 1
                mask = full ^ bit       # all except diagonal
                rows[x] = mask
                cols[x] = mask
                for i in range(n):
                    if i != x:
                        rows[i] |= bit
                        cols[i] |= bit
            else:
                rows[x] = 0
                cols[x] = 0
                m = ~bit
                for i in range(n):
                    if i != x:
                        rows[i] &= m
                        cols[i] &= m

        elif t == 4:
            # remove edge x -> y in visible graph
            x = next(it)
            y = next(it)
            r, c = (x, y) if not rev else (y, x)
            want = inv
            if want:
                rows[r] |= 1 << c
                cols[c] |= 1 << r
            else:
                rows[r] &= ~(1 << c)
                cols[c] &= ~(1 << r)

        elif t == 5:
            rev ^= 1
```

```
        else:  # t == 6
            inv ^= 1

    mask_all = (1 << n) - 1
    src = cols if rev else rows

    pow7 = [1] * n
    for i in range(1, n):
        pow7[i] = (pow7[i - 1] * 7) % MOD

    out = [str(n)]

    for i in range(n):
        raw = src[i] & mask_all
        adj = ((~raw) & mask_all) if inv else raw
        adj &= ~(1 << i)   # no self-loops in the actual graph

        deg = adj.bit_count()

        h = 0
        k = 0
        x = adj
        while x:
            lsb = x & -x
            v = lsb.bit_length() - 1
            h = (h + pow7[k] * v) % MOD
            k += 1
            x -= lsb

        out.append(f"{deg} {h}")

    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    solve()
```

It still gives Time Limit Exceeded

Thought for 29m 18s ›

A fast way is to process everything into a **hidden raw graph** and keep transpose/complement only as two lazy flags.

For any moment:

- `rev = 0/1`: whether the visible graph is the transpose of the raw one
- `inv = 0/1`: whether the visible graph is the complement of the raw one

So an update on the visible graph becomes a simple update on the raw graph.

The key offline observation is this:

For a pair of vertices `(u, v)`, its final raw value is determined only by the **latest** among:

- the moment this pair first came into existence
  - initial graph for initial vertices
  - the birth of the later-added vertex otherwise
- the latest "delete all incident edges" of `u`
- the latest "delete all incident edges" of `v`
- the latest explicit add/remove of that ordered raw edge

Because `Q <= 2000`, we can store these event times, then reconstruct each final row as a **bitset** using a handful of big-int operations, and patch only the sparse explicit edge updates.

That avoids `O(V)` work per query and is fast enough in Python.

```
import sys

MOD = 10**9 + 7
MASK16 = (1 << 16) - 1


def solve() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    it = iter(data)

    n0 = next(it)
    e = next(it)
    q = next(it)
    maxn = n0 + q

    # Initial graph (only among initial vertices).
    init_rows = [0] * n0
    init_cols = [0] * n0
    for _ in range(e):
        a = next(it)
        b = next(it)
        init_rows[a] |= 1 << b
        init_cols[b] |= 1 << a

    # For added vertices:
```

```
    # bt[v] = query time when v was added (0 for initial vertices)
    # bv[v] = raw value assigned between v and all older vertices at birth
    bt = [0] * maxn
    bv = [0] * maxn

    # Latest clear (delete all incident edges) per vertex:
    # ct[v] = latest clear time, cv[v] = raw value assigned by that clear
    ct = [-1] * maxn
    cv = [0] * maxn

    # Latest explicit raw edge assignments.
    edge_rows = [dict() for _ in range(maxn)]  # edge_rows[a][b] = (time<<
    edge_cols = [dict() for _ in range(maxn)]  # transpose view of the sam

    # exact0[t] / exact1[t]: vertices whose latest clear is exactly at tim
    # and assigns raw 0 / raw 1.
    exact0 = [0] * (q + 1)
    exact1 = [0] * (q + 1)

    # add_before[t] = number of type-1 queries before time t
    add_before = [0] * (q + 1)

    n = n0
    adds = 0
    rev = 0
    inv = 0

    for t in range(1, q + 1):
        add_before[t] = adds
        typ = next(it)

        if typ == 1:
            bt[n] = t
            bv[n] = inv
            n += 1
            adds += 1

        elif typ == 2:
            x = next(it)
            y = next(it)
            a, b = (y, x) if rev else (x, y)
            enc = (t << 1) | (1 ^ inv)
            edge_rows[a][b] = enc
            edge_cols[b][a] = enc

        elif typ == 3:
            x = next(it)
            old = ct[x]
            if old > 0:
                bit = 1 << x
                if cv[x]:
                    exact1[old] &= ~bit
```

```
                else:
                    exact0[old] &= ~bit

            ct[x] = t
            cv[x] = inv
            if inv:
                exact1[t] |= 1 << x
            else:
                exact0[t] |= 1 << x

        elif typ == 4:
            x = next(it)
            y = next(it)
            a, b = (y, x) if rev else (x, y)
            enc = (t << 1) | inv
            edge_rows[a][b] = enc
            edge_cols[b][a] = enc

        elif typ == 5:
            rev ^= 1

        else:  # typ == 6
            inv ^= 1

    bt = bt[:n]
    bv = bv[:n]
    ct = ct[:n]
    cv = cv[:n]
    edge_rows = edge_rows[:n]
    edge_cols = edge_cols[:n]

    # after0[t] / after1[t]:
    # vertices whose latest clear time is > t and assigns raw 0 / raw 1.
    after0 = [0] * (q + 1)
    after1 = [0] * (q + 1)
    run0 = 0
    run1 = 0
    for t in range(q, -1, -1):
        after0[t] = run0
        after1[t] = run1
        if t > 0:
            run0 |= exact0[t]
            run1 |= exact1[t]

    all_mask = (1 << n) - 1

    # Bitset of later vertices whose birth raw value is 1.
    later1_all = 0
    # Bitsets of later vertices whose latest clear happened after their bi
    later_after_birth0 = 0
    later_after_birth1 = 0
```

```
    for v in range(n0, n):
        bit = 1 << v
        if bv[v]:
            later1_all |= bit
        if ct[v] > bt[v]:
            if cv[v]:
                later_after_birth1 |= bit
            else:
                later_after_birth0 |= bit

    # Final visible rows come from raw rows or raw cols depending on final
    row_ops = edge_cols if rev else edge_rows
    init_base = init_cols if rev else init_rows

    # Powers of 7.
    L = max(n, 16) + 1
    pow7 = [1] * L
    for i in range(1, L):
        pow7[i] = (pow7[i - 1] * 7) % MOD

    # 16-bit chunk DP for hashing rows quickly.
    cnt = [0] * (1 << 16)
    A = [0] * (1 << 16)  # sum 7^k for set bits in the chunk
    B = [0] * (1 << 16)  # sum 7^k * local_position
    for p in range(1, 1 << 16):
        b = p.bit_length() - 1
        msb = 1 << b
        q2 = p ^ msb
        c = cnt[q2]
        cnt[p] = c + 1
        A[p] = (A[q2] + pow7[c]) % MOD
        B[p] = (B[q2] + pow7[c] * b) % MOD

    chunks = (n + 15) >> 4
    out = [str(n)]

    for i in range(n):
        # Base row: state coming only from initial graph / births.
        if i < n0:
            row = init_base[i] | later1_all
        else:
            row = (((1 << i) - 1) if bv[i] else 0) | (later1_all & ~((1 <<
        row &= ~(1 << i)

        Ti = ct[i]
        bi = bt[i]

        # Row-clear of i overrides columns whose pair existed before Ti,
        # unless that column has a later clear.
        if Ti > bi:
            cut = n0 + add_before[Ti]  # vertices born before Ti
            if cut > n:
```

```
            cut = n
        m = ((1 << cut) - 1) & ~(1 << i) & ~(after0[Ti] | after1[Ti])
        if cv[i]:
            row |= m
        else:
            row &= ~m

    # Column-clears overriding from the left side j < i.
    thr_left = Ti if Ti > bi else bi
    if thr_left < 0:
        thr_left = 0
    m1 = ((1 << i) - 1) & after1[thr_left]
    m0 = ((1 << i) - 1) & after0[thr_left]

    # Initial vertices on the right, only if i itself is initial.
    if i < n0:
        right_init = ((1 << n0) - 1) ^ ((1 << (i + 1)) - 1)
        thr_init = Ti if Ti > 0 else 0
        m1 |= right_init & after1[thr_init]
        m0 |= right_init & after0[thr_init]

    # Later vertices on the right:
    # split into those born before Ti and those not.
    start = n0 if i < n0 else i + 1
    cut = n0 + add_before[Ti] if Ti > 0 else n0
    if cut > n:
        cut = n

    if cut > start:
        mid = ((1 << cut) - 1) ^ ((1 << start) - 1)
        m1 |= mid & after1[Ti]
        m0 |= mid & after0[Ti]

    st = start if start > cut else cut
    if st < n:
        aft = all_mask ^ ((1 << st) - 1)
        m1 |= aft & later_after_birth1
        m0 |= aft & later_after_birth0

    row |= m1
    row &= ~m0

    # Patch sparse explicit raw edge operations.
    for j, enc in row_ops[i].items():
        te = enc >> 1
        th = bt[i if i > j else j]  # pair birth time
        cti = ct[i]
        if th < cti:
            th = cti
        ctj = ct[j]
        if th < ctj:
            th = ctj
```

```
            if te > th:
                if enc & 1:
                    row |= 1 << j
                else:
                    row &= ~(1 << j)

        row &= ~(1 << i)

        # Apply final complement lazily.
        if inv:
            row = (all_mask ^ (1 << i)) ^ row

        deg = row.bit_count()

        # Fast hash using 16-bit chunks.
        h = 0
        used = 0
        x = row
        base = 0
        for _ in range(chunks):
            p = x & MASK16
            if p:
                h = (h + pow7[used] * ((base * A[p] + B[p]) % MOD)) % MOD
                used += cnt[p]
            x >>= 16
            base += 16

        out.append(f"{deg} {h}")

    sys.stdout.write("\n".join(out))


if __name__ == "__main__":
    solve()
```

Why this passes:

- queries are processed in `O(E + Q)`
- final reconstruction uses Python big-int bitsets, so each row is built with a small number of C-level bit operations
- explicit edge updates are sparse: at most `Q`
- hashing is accelerated with 16-bit chunk DP instead of iterating every neighbor one by one

If you want, I can also give a shorter version without the chunk-hash optimization.